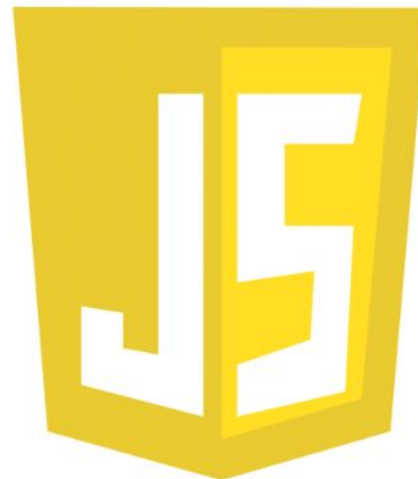


Programming 1

100 poäng, Kurskod: prrpr01



JavaScript



Mary Abeysekera
Pierre Gergi

Mål

Denna genomgång behandlar följande:

- Skillnader mellan Python & Javascript
- Grundläggande syntax i Python
- Övningar





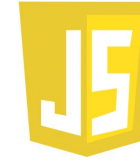
Python	JavaScript
Backend-utveckling	Frontend-utveckling
Enkelt och lättläst språk	Körs främst i webbläsare
Vanligt inom AI, dataanalys och automatisering	Vanligt för interaktiva webbplatser och webbappar
Används mycket på serversidan och i skript	Kan användas både i webbläsaren och på serversidan (t.ex. med Node.js)
Betonar läsbar kod	Byggt för att göra webbsidor dynamiska
Tillräckligt snabbt för de flesta uppgifter men används ofta tillsammans med optimerade bibliotek för tunga beräkningar.	Ofta snabbare för webbapplikationer eftersom det körs direkt i webbläsaren.



```
if x > 10:
    print("Stort")
else:
    print("Litet")
```

```
name = "Alice"
age = 25
```

JavaScript



```
if (x > 10) {
    console.log("Stort");
} else {
    console.log("Litet");
}
```

```
const name = "Alice";
let age = 25;
```



```
print("Hej")
```

```
for i in range(5):  
    print(i)
```

```
# Detta är en kommentar
```

JavaScript



```
console.log("Hej");
```

```
for (let i = 0; i < 5; i++) {  
    console.log(i);  
}
```

```
// Detta är en kommentar
```



```
function greet(name) {  
    return `Hej ${name}`;  
}
```

JavaScript



```
function greet(name) {  
    return `Hej ${name}`;  
}
```



Grunder i Python

Assembleringspråk

- I datorernas barndom var man tvungen att skriva program i denna form.

101000101001101 1001110110110101- binär form

- En 1 eller 0 är en bit - 8 bitar: 1 byte

- t.ex. **ENIAC** (Electronic Numerical Integrator And Computer) som blev klar 1946. var man till och med tvungen att programmera genom att ställa in ett antal knappar & kablar för hand.
- Enklare blev det när man började använda s.k. assembleringspråk.
D.V.S. Assembleringspråk är en enkel typ av (Low-level language / Lågnivåspråk)

- Då kunde man skriva programmet som vanlig text.
- **t.ex.** `LOAD A, ADD B, MOV C`
- Programmet måste först översättas till maskinkod. Detta görs av ett speciellt översättningsprogram.
- Även om det är mycket lättare att skriva program i ett assembleringsspråk än i maskinkod är det fortfarande mycket komplicerat.
- Nästa steg i utvecklingen var att man konstruerade mer avancerade programspråk, s.k. högnivåspråk (**High level language**).

Högnivåspråk

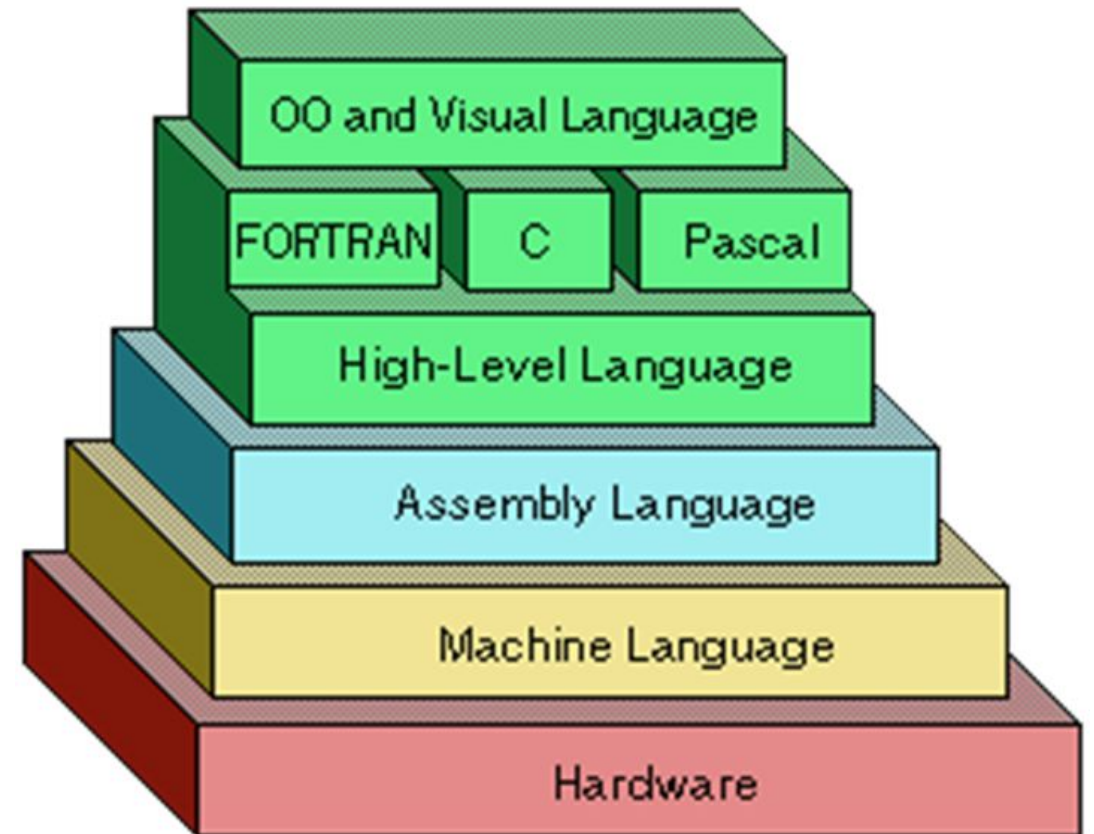
- Ett högnivåspråk är mer anpassat till det mänskliga sättet att uttrycka sig än till datorns maskinkod.
- I ett högnivåspråk skrivs programmen i en sorts halvengelska.
- I ett högnivåspråk skriver man programmet i en form som är lättare att förstå för människor.
- För att ett program ska kunna exekveras i datorn måste det finnas som maskinkod (nollor och ettor). Då därför måste översättas till maskinkod genom att använda Interpreter (Interpretator) eller Compiler (Kompilator).

Högnivåspråk

- Enkelt för programmerare att förstå
- Innehåller vanliga engelska ord

Lågnivåspråk

- Datorns egna språk
- Binära tal (ettor och nollor)



C-kod

```
#include <stdio.h>

#include <stdlib.h>

int main(void) {

    puts("!!!Hello World!!!"); /* prints !!!Hello World!!
    ! */

    return EXIT_SUCCESS;

}
```

Assembler-kod

```
.Ltext0:
    .section      .rodata
.LC0:
    .string "!!!Hello World!!!"

    .text
    .globl main
main:
.LFB0:
    .cfi_startproc
0000 55      pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
0001 4889E5  movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    movl    $.LC0, %edi
    call    puts
    movl    $0, %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE0:
.Letext0:
```

Interpretatorer och Kompilatorer

- Enkelt uttryckt så kan vi säga att interpreters *och* compilers är översättare.
- De tar ett program skrivet i ett programmeringsspråk så som Python eller Java och översätter det till maskinkod som datorn sedan kan utföra.
- En interpretator å andra sidan fungerar mer som en tolk, som översätter rad för rad i och med att programmet körs.
- En kompilator tar ett helt program och översätter det till en körbar fil (t.ex. exe eller app).

Interpretatorer

Kompilatorer

- Båda har sina för- och nackdelar.

Kompilatorer

- + Snabb exekvering (eftersom översättningen redan är gjord).
- + Bra för stora program där prestanda är viktigt
- + Fångar ofta många fel redan innan programmet körs
- Tar tid att kompilera
- Mindre flexibel för snabba tester eller interaktiv körning

Interpreter

- + Lätt att testa och ändra kod snabbt
- + Bra för nybörjare och inläring
- + Bättre för interaktiv användning
- Går oftast långsammare
- Felen hittas ibland först när koden körs

-
- Exempel på språk som ofta interpreteras är: Python, JavaScript, Ruby, och olika kommandotolkar i operativsystem.
 - Exempel på språk som använder kompilator är: Java, C, C++, C# och vissa varianter av Pascal.

Debugging är en process att felsöka och åtgärda fel (errors) i ett datorprogram.

Programmering i Python

- Python är ett populärt högnivåspråk med flera ändamål och enkla syntaxer.
 - Många anser att språkreglerna i Python är relativt enkla och att det därför, även för en nybörjare, inte är så svårt att komma igång.
 - Denna kurs hjälper dig att ta första steget in i den spännande världen som är kodning.
-
- Python lanserades av Guido van Rossum år 1991.
 - Den har filändelsen ".py".



-
- Att börja med Python behöver man INTE ha några tidigare kunskaper i programmering.
 - Att programmera är ett praktiskt arbete, även om det förstås också krävs mycket tankeverksamhet.
 - Målsättningen är att ge dig en stabil grund att stå på, så att du sedan kan fortsätta och studera mer avancerad programmering.
 - Denna kurs ger dig också allmänna kunskaper i programmering som du har nytta av även om du senare programmerar i a



Python Styrkor:

En styrka med detta språk är att det kan användas för att utveckla flera olika slag av applikationer.

För avancerade
matematiska
beräkningar

Dataanalys &
AI

Cybersäkerhet

För grafik och för
webbprogrammeri
ng.

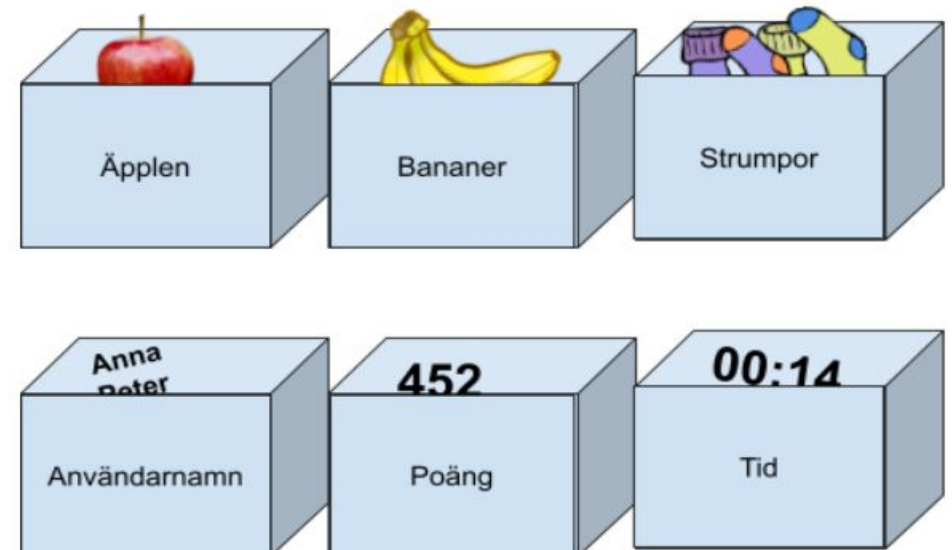
Spelutveckling

Variabler och typer

- I programmering är en variabel en minnesplats där man lagrar ett tal eller en text (sträng).
- Beräkningar utförs stegvis och man sparar mellanresultat för att kunna användamdem senare. Då använder man variabler.



`skola="Jensen"`



Datatyper

- En variabel kan innehålla data av olika slag.
- Vilken typ den får beror på vad man tilldelar den.

○ String (text)	"Hej" "123"
○ Integer (heltal)	23
○ Float (decimaltal)	34.6
○ Boolean	True/ False

Casting

Om man vill omvandla sin variabel eller värde från en typ till en annan så kan man casta.

Ex:

`int()` kan casta ett flyttal eller sträng till ett heltal `int("3")`
`float()` kan casta ett heltal eller sträng till ett flyttal `float("3.1")`
`str()` kan casta ett heltal eller sträng till ett sträng `str(3) -> "3"`

Aritmetiska uttryck

- Precis som i vanlig matematik kan man bilda uttryck som innehåller de fyra räknesätten.
- Addition: + Multiplikation: *
- Subtraktion: - Division: /
- Tecknen +, -, * och / kallas aritmetiska operatorer.
- Potenser skriver man med dubbla ** operatör t.ex 2^2 skrivs **2**2**
- Funktionen "pow" (power) används för potensberäkningar.
I exemplet returneras värdet 2 upphöjt till 4 (samma som $2 * 2 * 2 * 2$)

```
x = pow(2, 4)
print(x)
```

Fördjupning

Med operatören ****** kan man beräkna en potens:

```
print (7**2)
```

49

Operatören **//** utför en division och skriver ut heltalsdelen av resultatet. Notera att operatören avrundar neråt:

```
print(7//2) 3 ELLER print(int(7/2)) 3
```

Om man vill beräkna kvadratroten av ett tal, kan man utnyttja att det är detsamma som att upphöja talet till 0,5:

```
print(49**0.5)
```

7

Matematikmodulen

Python har också en inbyggd modul som kallas för "math", som utökar listan över matematiska funktioner.

För att kunna använda den måste man importera matematikmodulen "math":

Metoden `math.sqrt()` returnerar till exempel kvadratroten av ett tal:

Ex.

```
import math
x = math.sqrt(64)
print(x)      8
```

```
import math
x = math.pi
print(x)
```


Kommentarer

För att göra program lättare att förstå kan man lägga in kommentarer.

- Python-interpretatorn hoppar över kommentarer och struntar i vad som står i dem.
- Att använda kommentarer i ett program är nödvändigt för den som ska läsa programkoden och eventuellt göra förändringar i den.
- Kommentarer t.ex. beskriver vad en funktion gör.
- I Python använder man tecknet # för att göra en rad till en kommentar.

Ex. # Detta är en kommentar

Skriva tal och text

Vi kan också skriva text och beräkningar på en och samma rad. Då använder vi **kommatecken** för att skilja mellan text och beräkningar.

Detta är en övning

```
print("Du ska betala:", 38 + 11 + 8, "kr.")
```

Du ska betala: 57 kr.

OBS!

- Lägg till en indentering (tab) genom att skriva `\t`.
- Hoppa ner till nästa rad (som enter) genom att skriva `\n`.

Ex.

```
print("Languages:\nPython\tJava\nJavaScript")
```

```
Languages:
```

```
Python  Java
```

```
JavaScript
```

Avrunda

Om du vill avrunda ett värde kan du använda kommandot `round ()`. Så här avrundar du talet 663.671245 till två decimaler.

```
print(round(663.671245, 2))  
663.67
```

```
print(round(663.991245))  
664
```

- Vi kan också börja programmet med att spara rektangelns bredd och längd i minnet. Då använder vi två variabler.

```
bredd = 3
längd = 4
print ("En rektangel har bredden", bredd, "cm och längden", längd, "cm.")
print ("Omkretsen är", (bredd + längd)*2, "centimeter.")
print ("Arean är", längd * bredd, "kvadratcentimeter.")
```

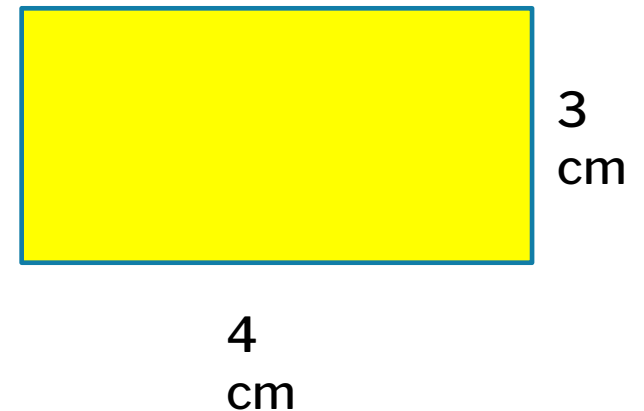
Exempel

En rektangel har bredden 3 cm och längden 4 cm.

Omkretsen är 14

Arean är 12

kvadratcentimeter.



Input

att mata in text och tal



Input - att mata in text och tal

- ✓ Med kommandot **input ()** låter man användaren mata in en sträng i programmet.

```
print("Vänligen fyll i ditt namn:")  
namn = input ()  
print("Trevligt att träffas",namn)
```

Eller

```
namn = input ("Vänligen fyll i ditt namn:")  
print("Trevligt att träffas",namn)
```

Exempel 

- När inmatningen är ett tal använder man kommandot `int ()` eller `float()`.

#Använd `int ()` när inmatningen är ett heltal.

```
ålder = int(input ("Hur många år är du?"))
```

#Använd `float()` när inmatningen är ett tal med decimaler.

```
längd = float (input ("Hur lång är du?"))
```

```
print("Din ålder är:" , ålder , "Din längd är:", längd)
```


- ✓ När man inte definierar datatypen tolkar programmet inmatningen som en text (sträng).

```
tal= input ("Skriv ett heltal:")  
print(5 * tal)
```

Skriv ett heltal: 8
88888



Resultatet blir 88888 när vi matar in talet 8, och inte det förväntade 40, beror på att programmet tolkar inmatningen som en sträng.

```
tal= int(input ("Skriv ett heltal:"))  
print(5 * tal)
```

Skriv ett heltal: 8
40

Exempel

```
namn= input ("Vad heter du?")  
print("Hej" , namn+"!")
```

OBS! För att undvika mellanslag runt en variabel som innehåller text, kan man använda plustecken **+** i stället för kommatecken **,**.

Vad heter du? Alice

Hej Alice!



Uppgift 1

-Skriv ett program där användaren:

a) skriver sin ålder och programmet anger åldern om **fem** år.

Exempel

Programmet ska hälsa : Skriv din ålder:

Output: Om fem år är du ___ år gammal.

b) anger sin längd i meter och programmet anger längden i centimeter.

Exempel

Programmet ska hälsa : Skriv din längd i meter:

Output: Du är alltså ___ cm.

Uppgift 2



- Skriv ett program som frågar efter användarens ålder och beräknar maxpulsen.

För att uppskatta din maxpuls kan du använda formeln:

$\text{maxpuls} = 220 - \text{ålder}$

Hur gammal är du?

Din maxpuls är ____ slag per minut

Exempel

For

Att upprepa kod



Python loops

For-att upprepa kod

- En av styrkorna med programmering är att man kan upprepa en bit kod ett bestämt antal gånger.
- När man upprepar en bit kod säger man att man gör en *slinga* eller en *loop*.

Upprepa med for

```
for n in range(5): # Efter ett for-kommando skriver man alltid ett
    print("Hej")    kolon.
```

Exempel 



- Om du tittar noga i programmet, ser du att den andra raden börjar lite längre in än den första. Detta kallas för en **indentering** eller ett **indrag**.

Hej

Hej

Hej

Hej

Hej

- Allt som är indenterat kommer programmet att upprepa. Det kallas för en **loop** eller för en **slinga**.

- Indraget görs med tabbtangenten eller med dubbla mellanslag.

For-Loopar med Stopp

```
for i in range(stopp):
```

Efter ett for-kommando skriver man alltid ett kolon.

```
    print(i)
```

Exempel

Exempel:

```
for i in range(5):  
    print(i)
```

0

1

2

3

4

Variabeln "i" börjar från 0

For-Loopar med Start & Stopp

```
for i in range(start, stopp):  
    print(i)
```

Exempel 

Exempel:

```
for i in range(1, 6): # Efter ett for-kommando skriver man alltid ett kolon.  
    print(i)
```

1

2

3

4

5

OBS! Programmet skriver ut alla positiva heltal mellan 1 och 5, och inte som man kan tro mellan 1 och 6.

Använd For-Loopar för beräkningar

Exempel:

```
summa = 0
for i in range(1,101):
    summa =summa + i
print("Summan av all tal mellan 1 och 100:", summa)
```

Programmet skriver ut summan av alla heltal mellan 1 och 100

```
summa =summa + i
```

kan också skrivas som

```
summa += i.
```

OBS! Man kan använda denna metod för alla operationer

`+=` , `-=` , `*=` , `/=`

For Loopar med Start, Stopp och Steg

```
for i in range(start, stopp, steg) :  
    print(i)
```

Exempel:

```
for i in range(0, 21, 5) :  
    print(i)
```

0

5

10

15

20

Programmet skriver ut alla var femte positivt heltal mellan 0 och 20



Uppgift 3

Vilket resultat kommer vi att få när vi implementerar detta program?

```
for n in range(1, 5):  
    print("Jensen!")
```



Exempel



Uppgift 4

- Skriv ett program som skriver ut alla heltal från och med

- a) 1 till och med 11
- b) 1 till och med 100
- c) -100 till och med 5
- d) 50 till och med -50

Exempel 



Uppgift 5

- Skriv ett program som skriver var 4:e tal mellan 16 och 2048 (inklusive).

Exempel 



- Skriv ett program som skriver ut de tio första talen i sjuans multiplikationstabell.

7

14

21

28

35

42

49

56

63

70

Exempel

Nästlade For Loopar

Att upprepa kod



Python loops

Nästlade For Loopar

```
for i in range(stopp):  
    for j in range(stopp):
```

Exempel:

```
for i in range(3):  
    for j in range(2):  
        print("Hej ")
```

```
print("\n")
```

Hej
Hej

Hej
Hej

Hej
Hej



- Skriv ett program som skriver ut multiplikationstabell. (Tabell 1 till tabell 10)

1
2
3
4
5
6
7
8
9
10

2
4
6
8
10
12
14
16
18
20

.

Exempel 



Jämförelseoperatorer

Operatorer (Compare

Operators)

När vi ställer upp villkor använder vi operatorer.

	Tecken i matematiken	Tecken i programmering
Mindre än	$<$	$<$
Mindre än eller lika med	$\leq$	$\leq$
Lika med	$=$	$==$
Större än eller lika med	$\geq$	$\geq$
Större än	$>$	$>$
Inte lika med	$\neq$	$\neq$

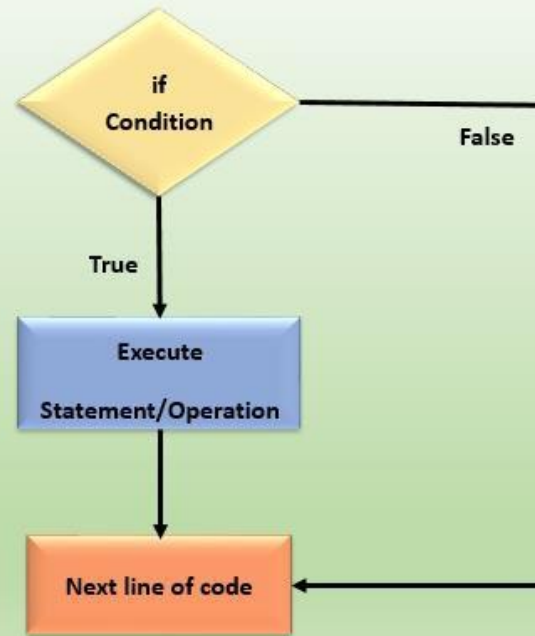
Jämförelser returnerar bool

```
print(5 > 3)           # True
print(10 == 2)         # False
print("hej" != "HEJ")  # True
```

If

att skriva
villkor

If Statement in Python



If- att skriva villkor

Ibland vill man att en viss bit kod ska köras, bara om ett visst villkor är uppfyllt. Då använder man kommandona *if*, *else* och *elif*.

Ex. Python if Statement

```
num = 3
if num > 0:
    print(num, "is a positive number.")
```

```
num = -1
if num > 0:
    print(num, "is a positive number.")
```

Exempel 

If- och else villkor

Ibland vill man att något ska hända om ett villkor annars händer något annat

Ex. Python if and else Statement

```
num = 3
if num > 0:
    print(num, "is a positive number.")
else :
    print(num, "is a negative number.")
```


If-, elif och else villkor

Ibland vill man att saker olika saker ska hända på grund av flera olika villkor (mer än 2 olika villkor).

Ex. Python if, elif, and else Statement

```
num = 3
if num > 0:
    print(num, "is a positive number.")
elif num < 0 :
    print(num, "is a negative number.")
else :
    print(num, "is 0")
```

Exempel 

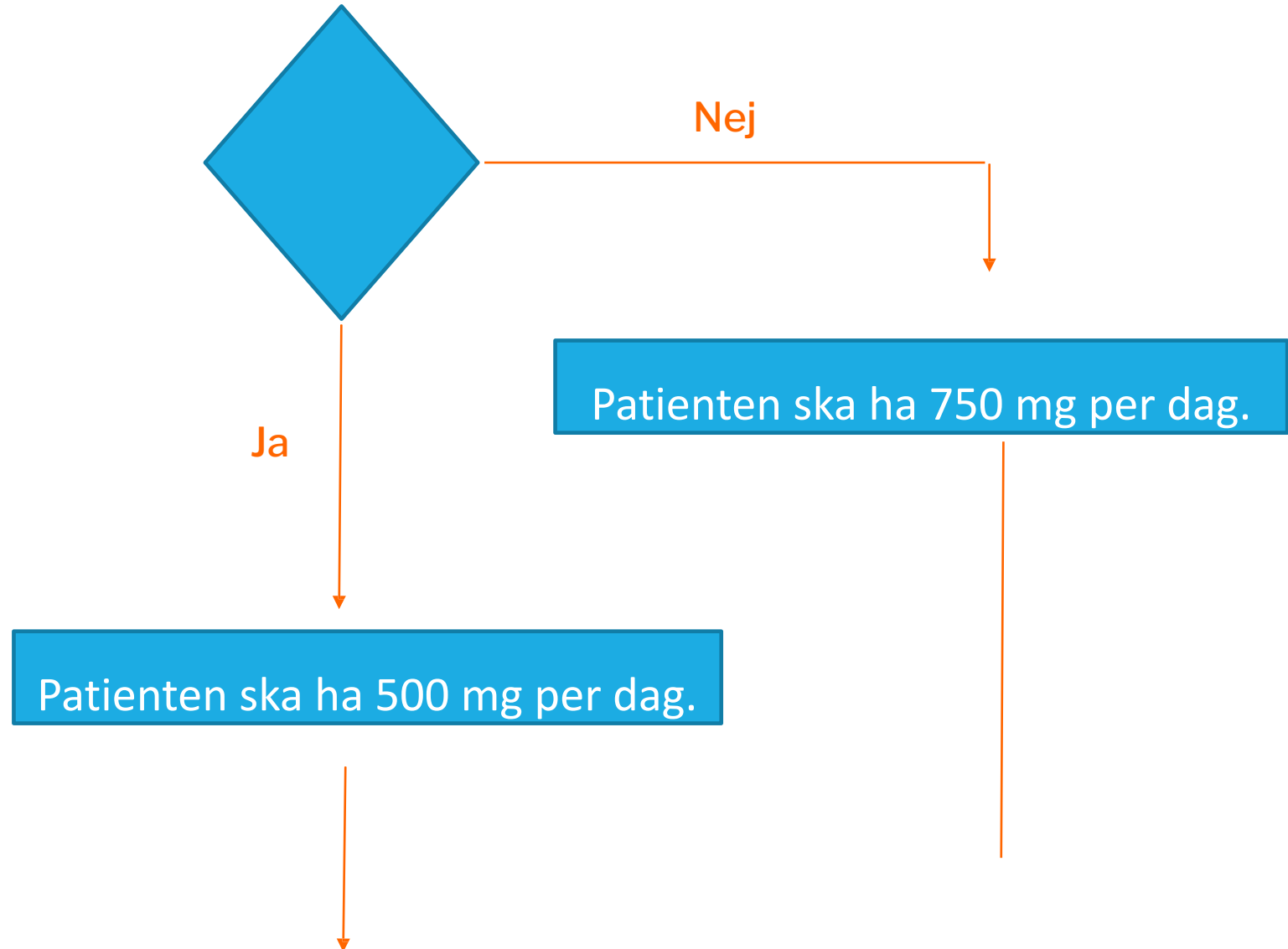
Ex. Ett program som tar hänsyn till patientens ålder respektive vikt och anger rätt dos medicin.

- Hur mycket medicin man ska få när man är sjuk, beror på om man är barn eller vuxen.

Vuxen	Barn
750 mg	500 mg

- Med detta program kan man ta reda på rätt dos.
- För att ta hänsyn till om patienten är barn eller vuxen, använder vi kommandona **if** och **else**

Är patienten ett barn?



Ex.

```
#Användaren får ange om patienten är ett barn.
```

```
svar = input("Är patienten ett barn? Y/N")
```

```
#Om svaret är "Ja" skriver programmet att patienten ska ha 500 mg per dag.
```

```
if svar == "Y":
```

```
    print("Patienten ska ha 500 mg per dag.")
```

```
#Annars skriver programmet att patienten ska ha 750 mg per dag.
```

```
else:
```

```
    print("Patienten ska ha 750 mg per dag. ")
```

Exempel 

Är patienten ett barn? Ja/Nej Y

Patienten ska ha **500** mg per dag.

- På den andra raden i programmet kontrollerar vi om variabeln svar är lika med "Y", Då använder vi **två** likhetstecken ==

elif Ibland beror dosen på hur mycket patienten väger.

Då kan programmet se ut så här:

```
vikt = int(input("Hur mycket väger patienten? Svara i kg. "))
if vikt < 20:
    print("Patienten ska ges 500 mg per dag.")
elif 20 <= vikt <= 40:
    print("Patienten ska ges 750 mg per dag.")
elif 41 <= vikt <= 60:
    print("Patienten ska ges 1000 mg per dag.")
else:
    print("Patienten ska ges 1500 mg per dag.")
```

Exempel 

Vikt	Dos
Mindre än 20 kg	500 mg per dag
20-40 kg	750 mg per dag
41-60 kg	1000 mg per dag
Mer än 60 kg	1500 mg per dag

Så här ser det ut när vi kör programmet.

Hur mycket väger patienten? Svara i kg. 32

Patienten ska ges 750 mg per dag.

- Eftersom det finns fler än två alternativ, använder vi kommandot **elif**. Man kan skriva flera elif-kommandon efter varandra.



Logiska operatorer

Logiska Operatorer

Används för att kombinera flera villkor

Operatorer	Namn	Förklaring	Exempel
and	och	Sant om båda villkoren är sanna.	$x > 5$ and $x < 10$
or	eller	Sant om minst ett av villkoren är sant.	$x < 5$ or $x > 10$
not	inte	Sant om uttrycket är falskt	not $x > 5$

Logiska Operatorer

Exempel:

x=4

print(6>5 and x==4) #True

print(3<5 or x==9) #True

print(not x==5) #True

Logiska Operatorer

Exempel:

x = 7

```
if x > 5 and x < 10:  
    print("x är mellan 5 och 10")
```

y = 3

```
if not y==5:  
    print("y är inte 5")
```

```
är_student = True  
är_pensionär = False
```

```
if är_student or är_pensionär:  
    print("Du får rabatt!")  
else:  
    print("Du får tyvärr ingen rabatt.")
```

While

Ett sätt att upprepa kod

```
while <expr>:  
    <statement(s)>
```

While - ett sätt att upprepa kod

- Tidigare använde vi en for-sats för att upprepa kod.
- Här introducerar vi ytterligare ett sätt att upprepa kod, nämligen med en while-sats.

while villkor :

kod
kod
kod

Exempel:

n=0
while n<10 :
 print(n)
 n=n+1

Skillnaden mellan **for** och **while**

while-satsen upprepar en bit kod så länge ett villkor är uppfyllt,
medan en
for-slinga upprepar koden ett bestämt antal gånger.

While

Exempel

- Med kommandot `while` kan man upprepa en bit kod, så länge ett villkor är uppfyllt.
- Det här programmet börjar med talet 1 och lägger till 2, så länge resultatet är mindre än 10.

```
n = 1
while n < 10:
    print(n)
    n=n+2
```

#Börja med talet `n` = 1 (initialvärde)
#Gör följande så länge talet `n` är mindre än 10:
#Skriv ut talet `n`
#Öka värdet av talet 2

- Som du ser skriver även det här programmet ut de udda talen mellan 1 och 10.
- När vi upprepar en kod, gör vi en slinga eller en loop. Du kan använda både **for** och **while** när du vill göra en slinga.

1
3
5
7
9

while True:

- Om man vill att programmet ska upprepa något oändligt många gånger, kan man använda kommandot **while True**. Det här programmet skriver en oändligt lång lista av udda tal:

```
n=1
while True:
    print(n)
    n = n + 2
```

Exempel 

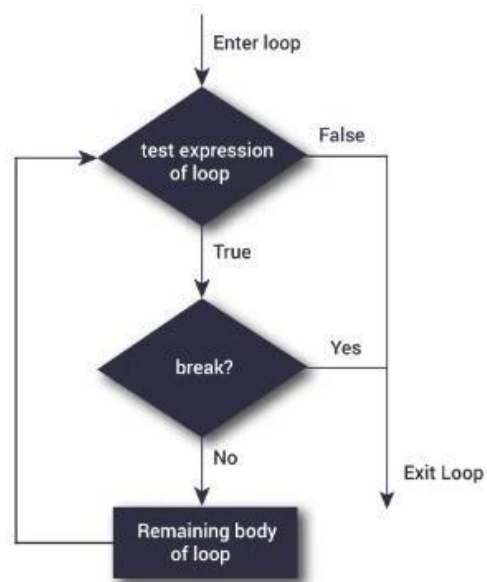
- Programmet upprepar **de inskjutna raderna** så länge villkoret är sant. Eftersom villkoret *True* alltid är sant, upprepas koden i all oändlighet.

OBS!

Om man får felmeddelanden, kan det bero på att man:

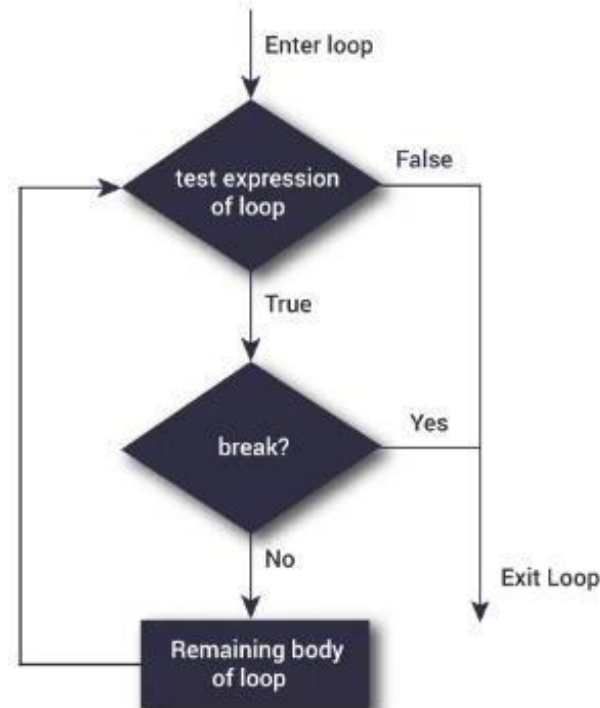
- glömt **kolon** efter raden med while-kommandot.
- missat **indenteringen**, dvs. missat att de rader som ska upprepas ska vara indragna.
- glömt definiera variabeln i **while-satsens villkor innan den används**. (initialvärde)

Break och Continue i loops (slingor)



Break

- Vi använder kommandot "**break**" för att avsluta slingan. Python kör INTE något efter ordet "break" i slingan, utan fortsätter med koden utanför slingan.
- Med andra ord kan kommandot "**break**" stoppa slingan även om **while-villkoret** är sant (**True**).



Vad gör detta program?

```
i = 0
while i < 5:
    print("This number is:", i)
    i = i + 1
    if i == 3 :
        break
print("The end!")
```

Exempel 

This number is 0

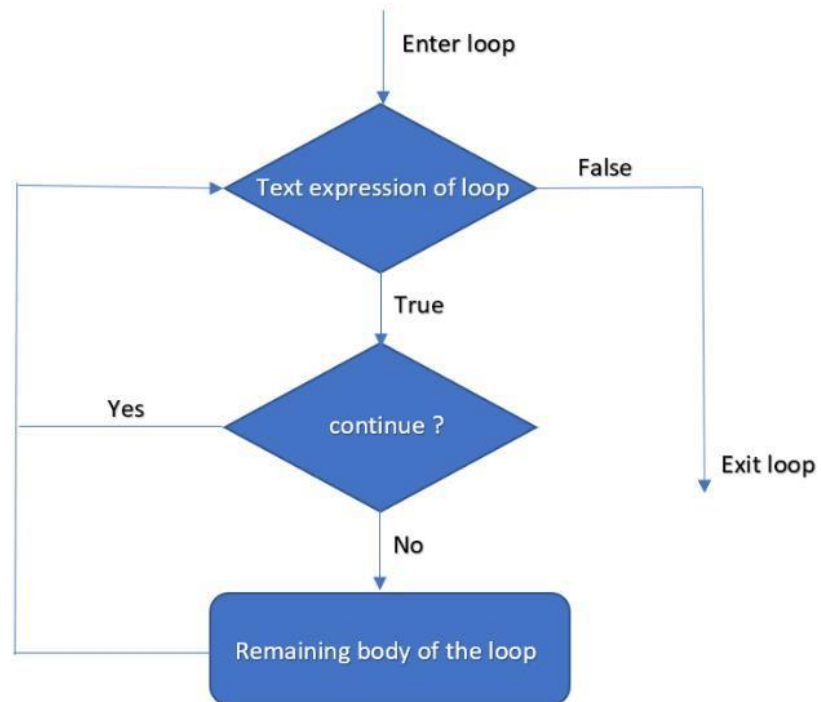
This number is 1

This number is 2

The end!

Continue

- Vi använder "**continue**" i slingan för att stoppa den nuvarande iterationen och fortsätta med nästa.



- Fortsätt till nästa iteration om "i" är 3:

```
i = 0
while i < 6:
    i = i + 1 # man kan skriva i += 1 istället för i = i + 1
    if i == 3:
        continue
    print(i)
```

N

Y

1
2
3
4
5
6

Exempel

- Fortsätt till nästa iteration om "i" är 4:

```
for i in range(1,7):
```

```
    if i ==4:  
        continue  
    print(i)
```

1

2

3

~~4~~

5

6

Exempel 



Listor (arrayer)

Vad är en lista?

- En lista (en array) är en samling av element.
- Istället för att bara spara ett värde i en variabel, kan vi med listor spara flera värden på samma gång.
- Listor skrivs med hjälp av klamrar []

Ex.

```
my_list = [5, 2, 8, 9, 3]
```

Element

my_list = [5, 2, "bok", 3, "penna"]

0 1 2 3 4

Index

Observera att indexeringen börjar från 0.

Skapa en listor:

Skapa tom lista:

```
listnamn= []
```

Skapa lista med element:

```
listnamn = [element, element, element, ...]
```

Exempel

Ex. # En lista används för att lagra flera värden i en enda variabel:

```
cars = ["Ford", "Volvo", "BMW"]  
print(cars)
```

```
['Ford', 'Volvo', 'BMW']
```

Välj element från listan:

Du refererar till lista element genom att hänvisa till indexnumret. Kom ihåg att listor i nollindexerad i python och vi börjar räkna element från 0.

`listnamn[index]`

Ex.

```
cars = ["Ford", "Volvo", "BMW"]  
print(cars[0])
```

Exempel 

Ford

len() - Längden av listan:

Använd **len()** metoden för att returnera antalet av element i en lista.

len(listnamn)

Ex.

```
cars = ["Ford", "Volvo", "BMW"]  
print(len(cars))
```

Exempel 

Loopa genom listan:

Du kan använda en for-loop för att gå igenom alla element i en lista.

```
for element in listnamn:           eller           for i in range (0, len(listnamn)):
    print(element)                      print(listnamn[i])
```

Ex..

```
cars = ["Ford", "Volvo", "BMW"]
for x in cars:
    print(x)
```

```
Ford
Volvo
BMW
```

Ex..

```
cars = ["Ford", "Volvo", "BMW"]
for i in range (0, len(cars)):
    print(cars[i])
```

```
Ford
Volvo
BMW
```

Exempel

append() – lägga till element på slutet av listan

Du kan använda metoden `append()` för att lägga till ett element i en lista i slutet av listan.

Ex.

```
cars = ["Ford", "Volvo", "BMW"]  
cars.append("Honda")  
print(cars)
```

```
['Ford', 'Volvo', 'BMW', 'Honda']
```

Exempel 

insert() – lägga till element på specifik plats

Metoden `insert()` lägger till ett element i en plats som du väljer. Du anger 2 värden. Första inmatningen är elementets index, och andra inmatningen är elements värde.

`listnamn.insert(index, element)`

Ex.

```
cars = ["Ford", "Volvo", "BMW"]
```

```
cars.insert(1, "Volkswagen")
```

"Volkswagen" infogas vid index 1 (andra plats). Alla element som kommer senare flyttas ett steg till höger.

```
print(cars)
```

Exempel 

```
["Ford", "Volkswagen", "Volvo", "BMW"]
```

pop()- ta bort element

Du kan använda `pop ()` metoden för att ta bort ett element från en lista.

`listnamn.pop(index)`

Ex.

```
cars = ["Ford", "Volvo", "BMW"]  
cars.pop(1) # I pop() metoden anges indexnumret.  
print(cars)
```

```
['Ford', 'BMW']
```

remove()- ta bort element

Du kan också använda metoden `remove()` för att ta bort ett element från en lista

Exempel

`listnamn.remove(element)`

Ex.

```
cars = ["Ford", "Volvo", "BMW"]  
cars.remove("Volvo") # I remove() metoden anges elementets värde.  
print(cars)
```

```
['Ford', 'BMW']
```

Exempel

Att ändra element i listan

Ex.

Exempel 

```
cars = ["Ford", "Volvo", "BMW",  
"Volkswagen"]  
cars[0] = "Mercedes"  
cars[3] = "Toyota"  
print(cars)
```

```
['Mercedes', 'Volvo', 'BMW', 'Toyota']
```

Clear()- ta bort alla element

Metoden `clear ()` tar bort alla element från en lista

Exempel 

Ex.

```
cars = ["Ford", "Volvo", "BMW"]  
cars.clear()  
print(cars)
```

```
[]
```



Dellistor

Dellistor

Du kan komma åt en del av en lista genom att använda operatorn `:` när du anger en listas index.

□ `lista[i:]` gör en dellista som och har med alla element fr.o.m. index `i` till slutet av listan.

```
lista = [1, 4, 9, 16, 25, 90]
```

```
print(lista[3:])
```

Exempel

[16, 25, 90]

`lista[:j]` gör en dellista som har med alla element från början av listan till index `j`.

Elementet med index `j` inkluderas inte.

```
lista = [1, 4, 9, 16, 25, 90]
```

```
print(lista[:3])
```

Exempel 

[1, 4, 9]

□ `lista[i:j]` gör en dellista som har med alla element fr.o.m. index i till index j.
Elementet med index j inkluderas inte.

```
lista = [1, 4, 9, 16, 25, 90]  
print(lista[2:4])
```

Exempel 

[9, 16]

- För att ta en kopia av en lista och spara det i ett annat namn skriver vi:

```
numbers = [1,2,3,4,5]
jensen = numbers[:]
print(numbers)
print(jensen)
```

Exempel 

Listan "jensen" är alltså en kopia av listan "numbers".

- Funktionerna `min ()` och `max ()` kan användas för att hitta det lägsta eller högsta värdet i en lista:

```
numbers = [1,2,3,4,5]  
print(min(numbers))  
print(max(numbers))
```

Exempel

1
5

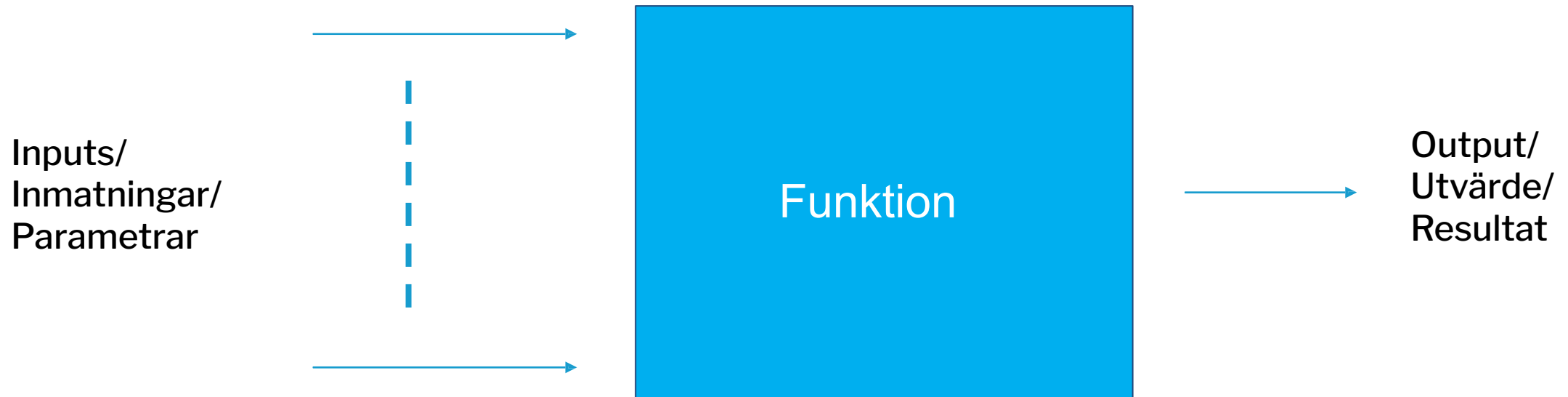
- Funktionen `sum ()` summerar alla element i listan. Alla element måste vara tal om denna funktion ska användas.

```
numbers = [1,2,3,4,5,6]  
print(sum(numbers))
```




Funktioner (Functions)

Repetition av funktioner



- En funktion är ett kodblock som bara körs när det anropas.
- Du kan skicka data, så kallade parametrar, till en funktion.
- En funktion kan returnera data som ett resultat.

Exempel:



Syntax för Python –funktion

Följande kodavsnitt visar den allmänna syntaxen för att definiera och anropa en funktion i Python:

```
def funktionsnamn(parametrar):  
    #Gör någonting  
    return resultat  
    # returnerar resultat till anropningsplatsen
```

Funktionsdefinition

```
#Kod utanför funktionen  
funktionsnamn(argument)    #anropningsplatsen  
funktionsnamn(argument)    #anropningsplatsen
```

Funktionsanrop

Exempel

parametrar!

```
def addera(tal1, tal2):  
    svar=tal1 + tal2 #Gör någonting  
    return svar  
    # returnerar resultat till anropningsplatsen
```

#Kod utanför funktionen

```
addera(58, 894) #anropningsplatsen, vi kallar på funktionen
```

argument!

Exempel

parametrar!

```
def addera(tal1, tal2):  
    svar=tal1 + tal2  #Gör någonting  
    print(svar)  
    # skriver ut svar, inget returneras till anropningsplatsen
```

#Kod utanför funktionen

```
addera(58, 894)  #anropningsplatsen
```

argument!

- Nu har vi definierat en egen funktion (**minBeräkning**). Den kan anropas och användas i resten av programmet (efter definitionen).

```
def minBeräkning (tal1, tal2):  
    svar = tal1 * tal2 + tal1  
    return svar
```

Exempel

```
# Kod utanför funktionen  
# Anropa och använd funktionen i den kommande delen av programmet.
```

```
print(minBeräkning(4,5))  
print(minBeräkning(8,9))
```

24
80

OBS! `minBeräkning(4,5)` och `minBeräkning(8,9)` ersätts av returvärdet.

Addera returvärden:

```
def minBeräkning (tal1, tal2):  
    svar = tal1 * tal2 + tal1  
    return svar
```

Man kan addera returvärden

```
y = minBeräkning(5,1) + minBeräkning(3,1)  
print(y)
```

Exempel

1
6

OBS! "minBeräkning(5,1)" och "minBeräkning(3,1)" ersätts
av returvärdet.

Valfri datatyp

Man kan skicka valfri **datatyp** (*sträng, nummer, lista, mm*) som parametrar till en funktion, och det kommer att behandlas som samma datatyp inuti funktionen. Samma gäller även returvärdet

Här skickar vi en **lista** som parameter och returnerar en **lista** som output

```
def my_function(food):  
    food.append("orange")  
    return food
```

```
print(my_function(["apple", "banana", "cherry"]))
```

Exempel

appl
banan
cherry
orange

Jobba själva!



Länk till övningar

Python övningar :

<https://docs.google.com/document/d/1JAHeF6T-QgDJ8yBPNakQkKKMn8faBF4IPC/Ten3ij8H8/edit?usp=sharing>